

Quick retrospective

What we've seen

Feature detection

Image

→ Interest points

Feature matching

Interest points

→ Corresponding point pairs

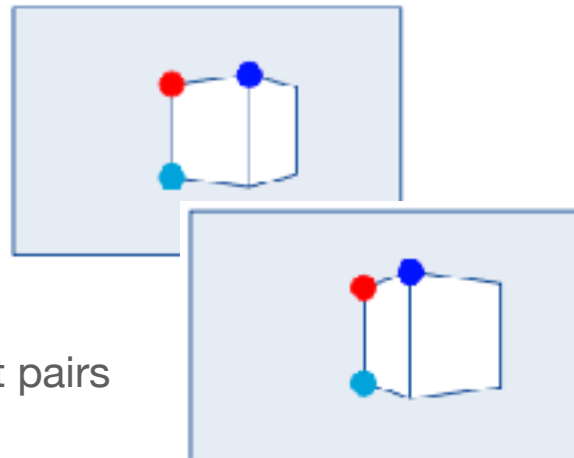
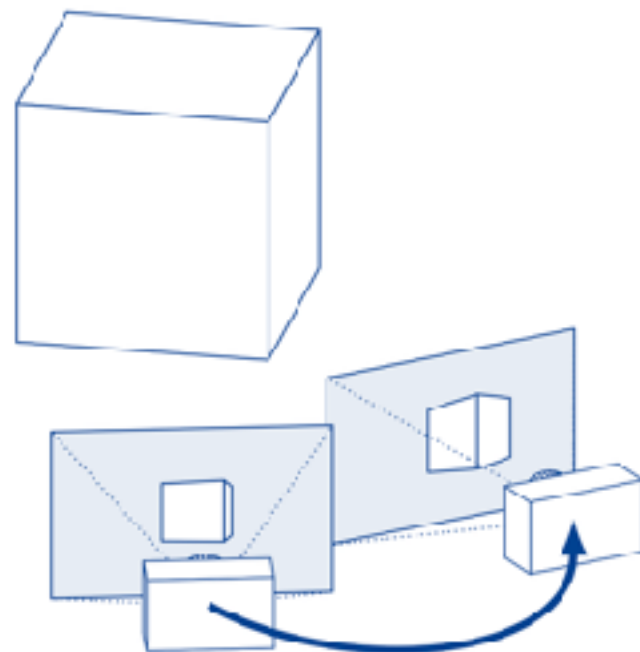
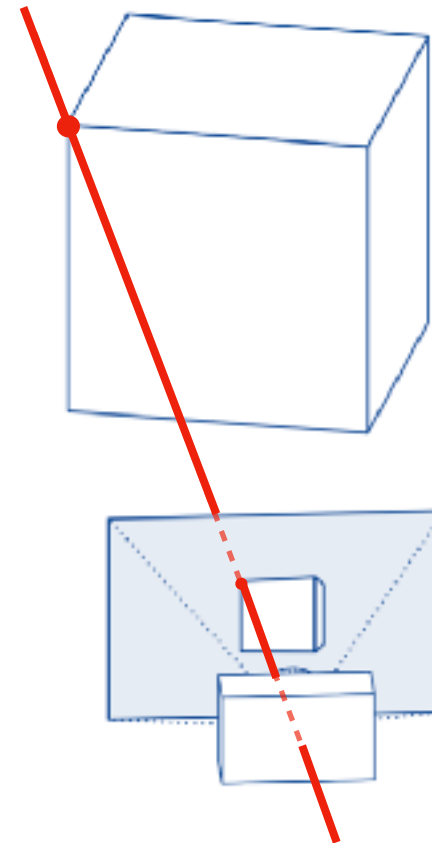


Image analysis



2-view geometry



Projection

3D point, camera projection matrix

→ Image point

Single-view geometry

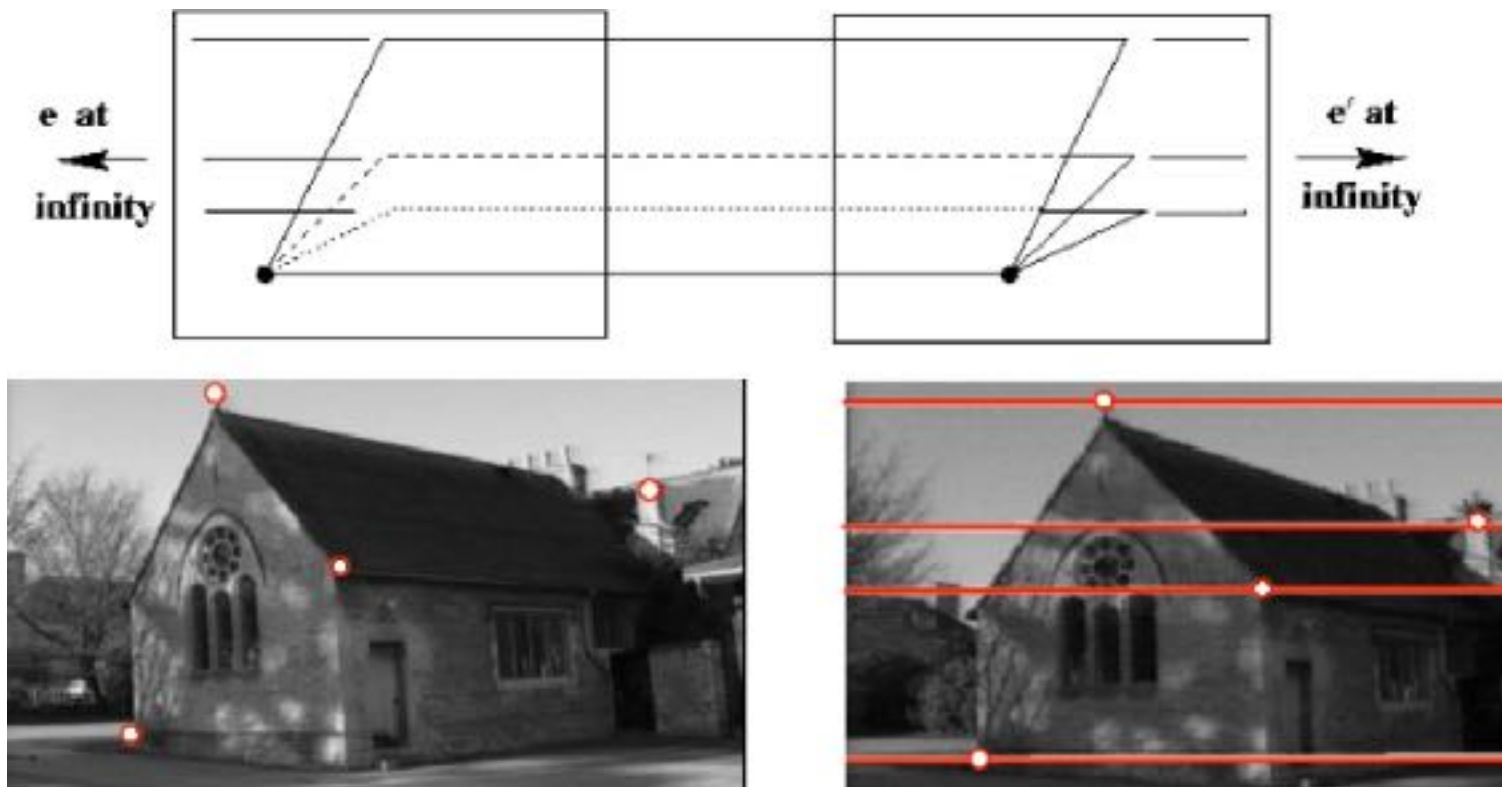
Epipolar geometry

Corresponding point pairs

→ relative orientation of cameras

Spoiler: Rectification (next lecture)

What if both cameras face exactly the same direction?



- Image planes are co-planar $\Rightarrow$ Epipoles at infinity, epipolar lines parallel
- Correspondences search along horizontal scanlines (simplifies implementation)

Rectification



- Correspondences are located on the **same image row** as the querypoint

What we've seen

Feature detection

Image

→ Interest points

Feature matching

Interest points

→ Corresponding point pairs

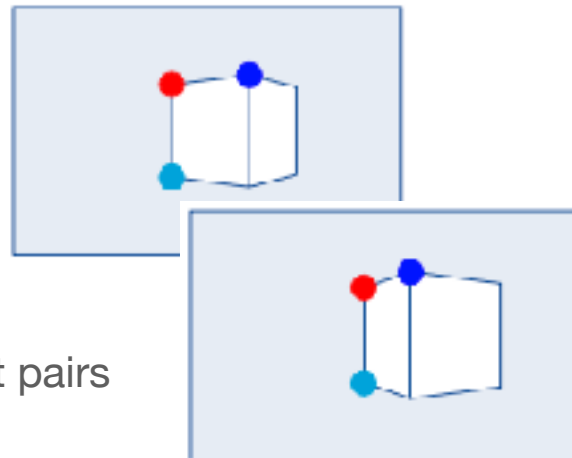
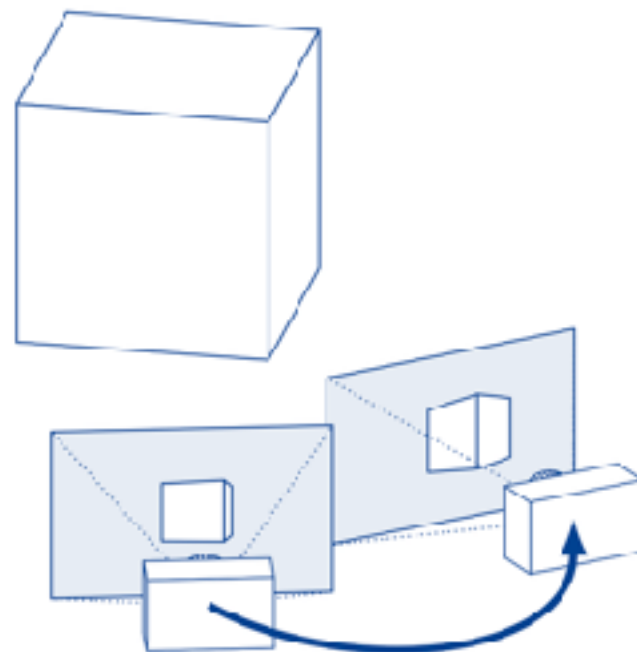
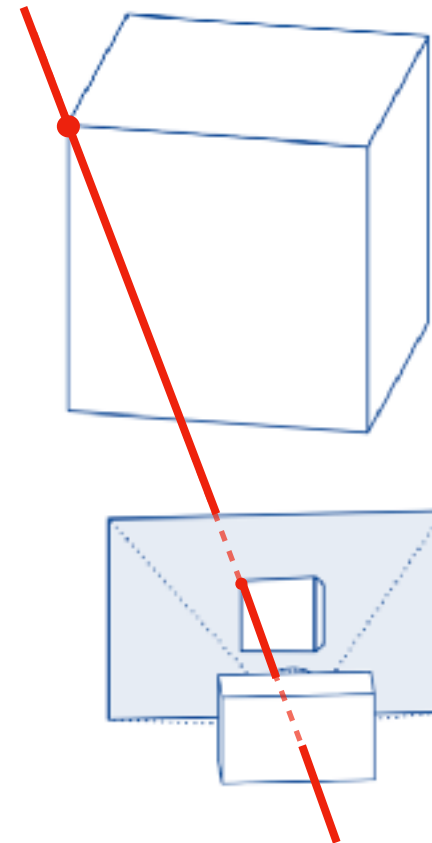


Image analysis



2-view geometry



Camera calibration

Image points, 3D points

→ projection matrix

Projection

3D point, camera projection matrix

→ Image point

Single-view geometry

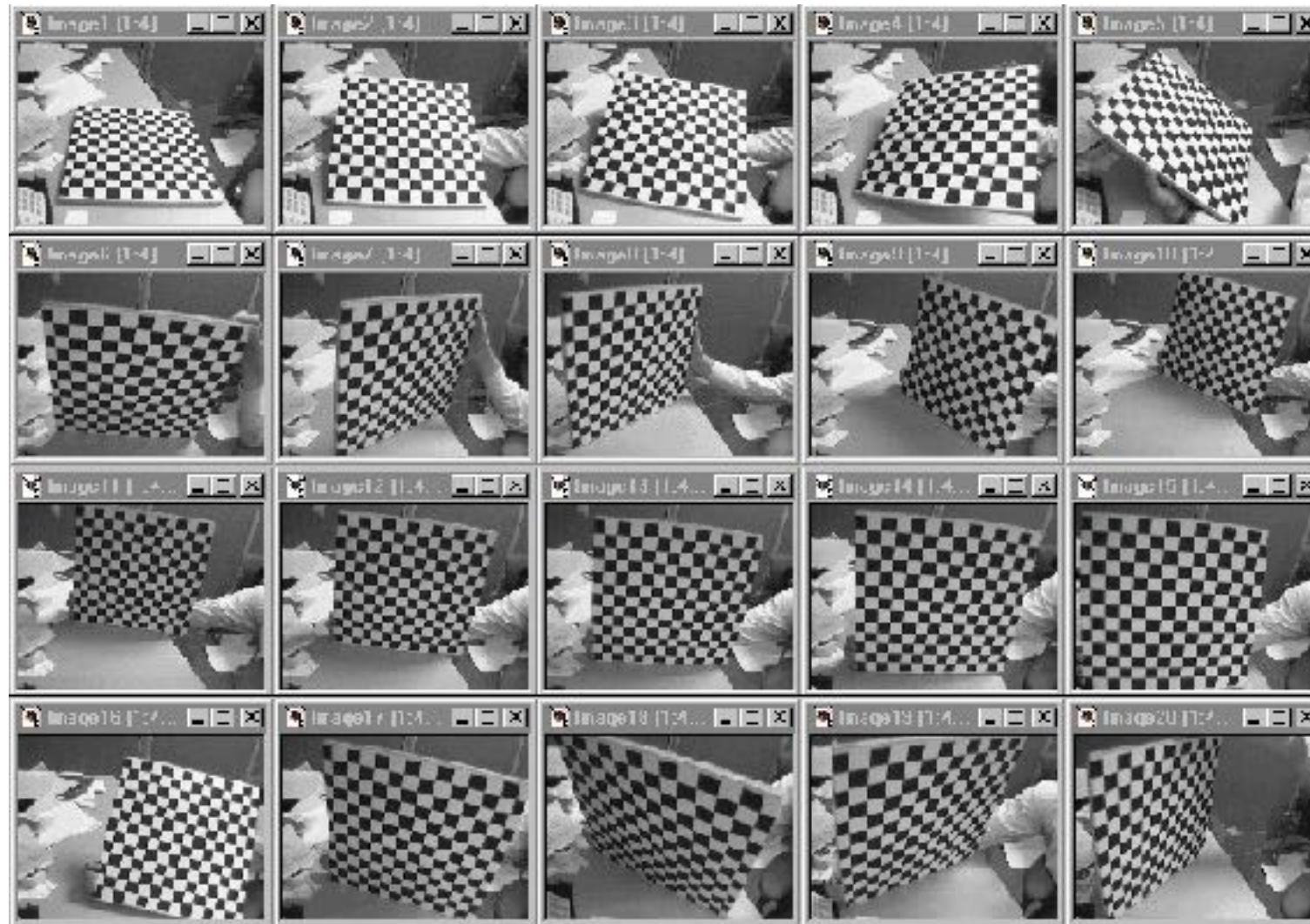
Epipolar geometry

Corresponding point pairs

→ relative orientation of cameras

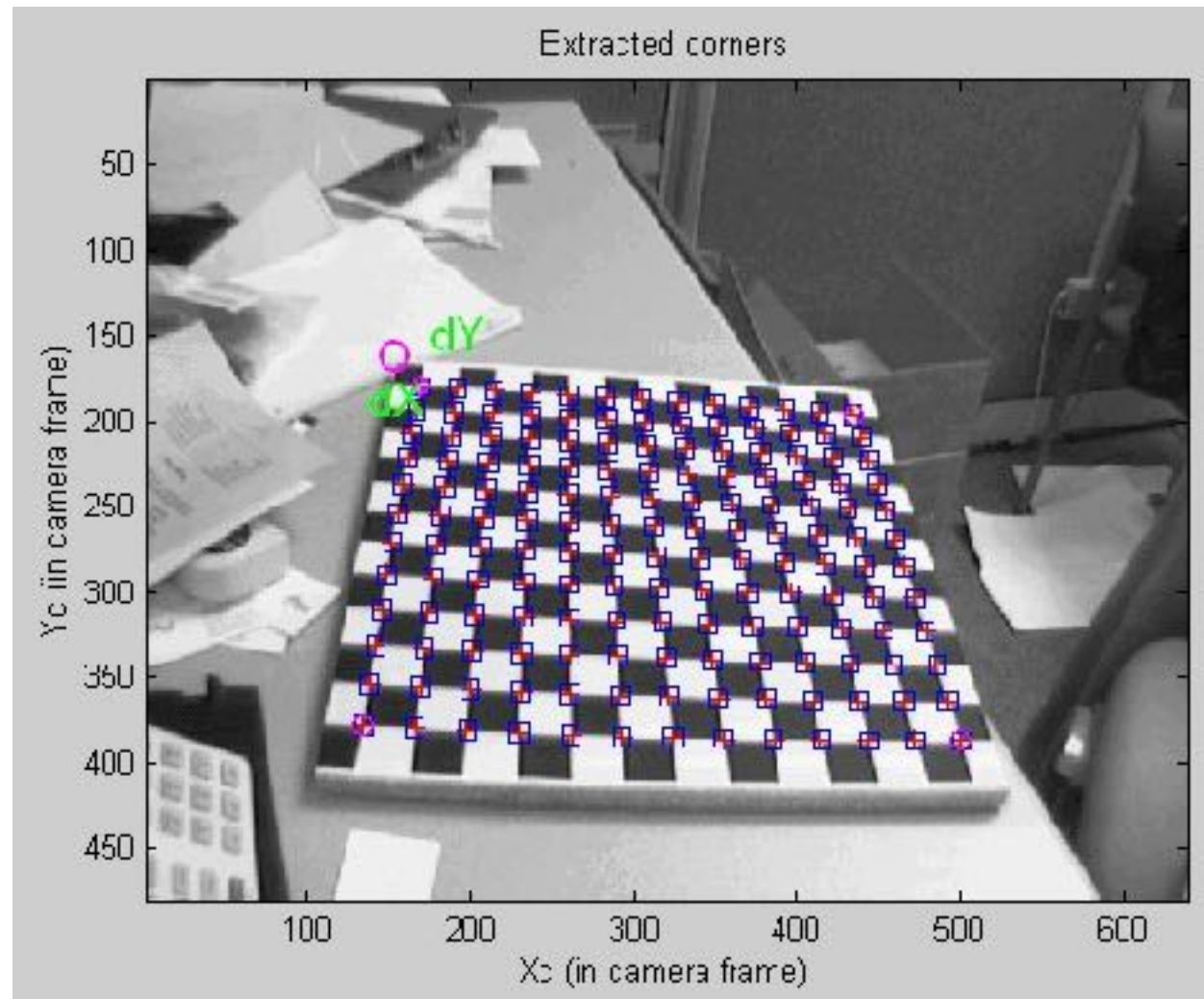
**Very briefly:
Camera calibration**

Camera calibration



- First, the known calibration target is **captured in different poses**

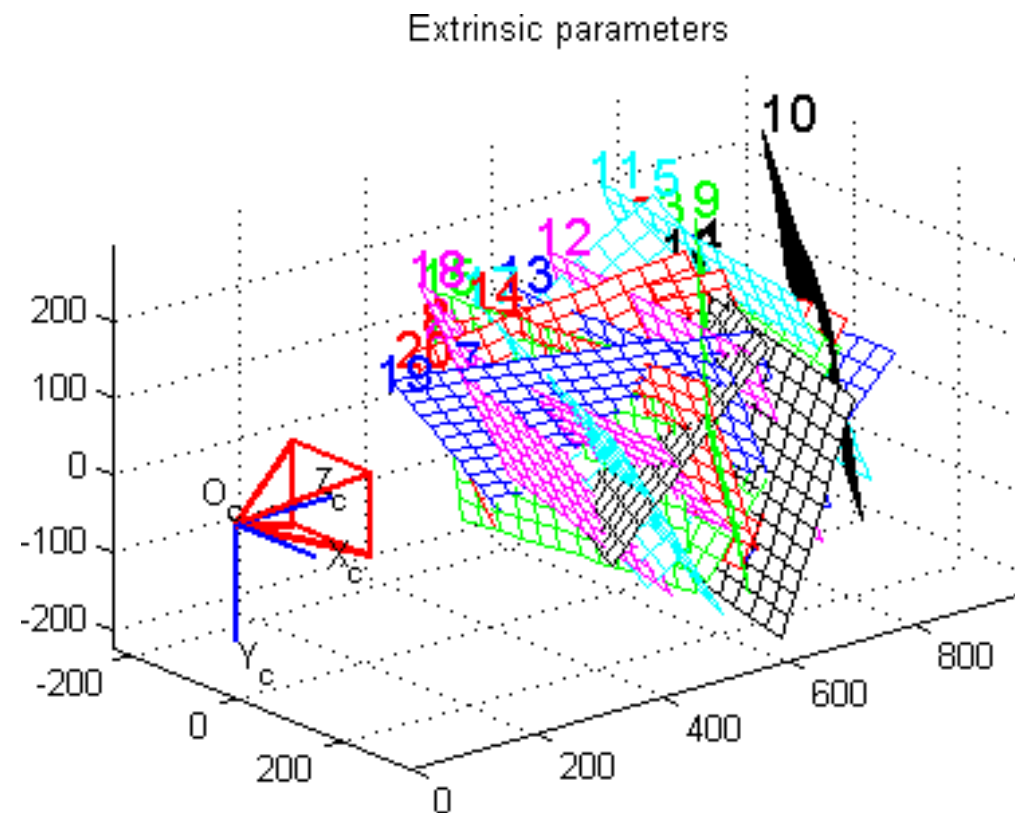
Camera calibration



- Second, **features** (e.g., corners) on the target are detected in the images

Camera Calibration

Zhang: A Flexible New Technique for Camera Calibration. TPAMI, 2000.



More details: Szeliski Book, Chapter 7.7

Finally, the **camera intrinsics and extrinsics (=poses)** are **jointly optimized**:

- ▶ **Closed-form solution** initializes all parameters except for distortion parameters
- ▶ **Non-linear optimization** of all parameters by minimizing reprojection errors

Recovering both cameras and scene geometry: Structure from Motion

Structure from motion

- Multi-view stereo assumes that cameras are calibrated
 - Extrinsic and intrinsic are known for all views
- How do we compute calibration if we don't know it?
- In general, this is called structure from motion

Large-scale structure from motion



- Dubrovnik, Croatia. 4,619 images (out of an initial 57,845).
- Total reconstruction time: 23 hours
- Number of cores: 352

Two views

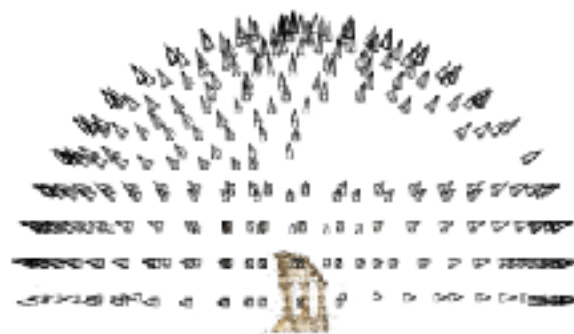


- Solve for Fundamental matrix / Essential matrix
- Factorize into intrinsics, rotation, and translation

What about more than two views?

- The geometry of three views is described by a $3 \times 3 \times 3$ tensor called the *trifocal tensor*
- The geometry of four views is described by a $3 \times 3 \times 3 \times 3$ tensor called the *quadrifocal tensor*
- After this it starts to get complicated. Instead, we explicitly solve for camera poses and scene geometry

Structure from motion



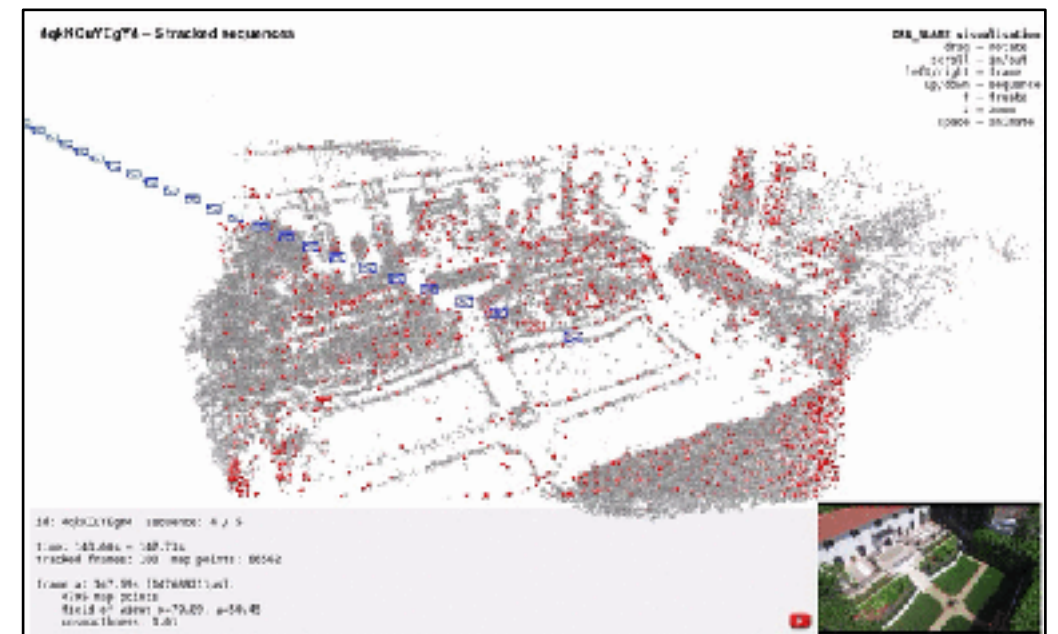
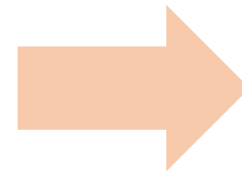
Reconstruction (side)



(top)

- Input: images with 2D points in correspondence
 $p_{i,j} = (u_{i,j}, v_{i,j})$
- Output
 - structure: 3D location $\mathbf{x}_i$ for each point p_i
 - motion: camera parameters $\mathbf{R}_j$, $\mathbf{t}_j$ & possibly $\mathbf{K}_j$
- Objective function: minimize *reprojection error*

Can also compute camera poses from video (often called Visual SLAM)



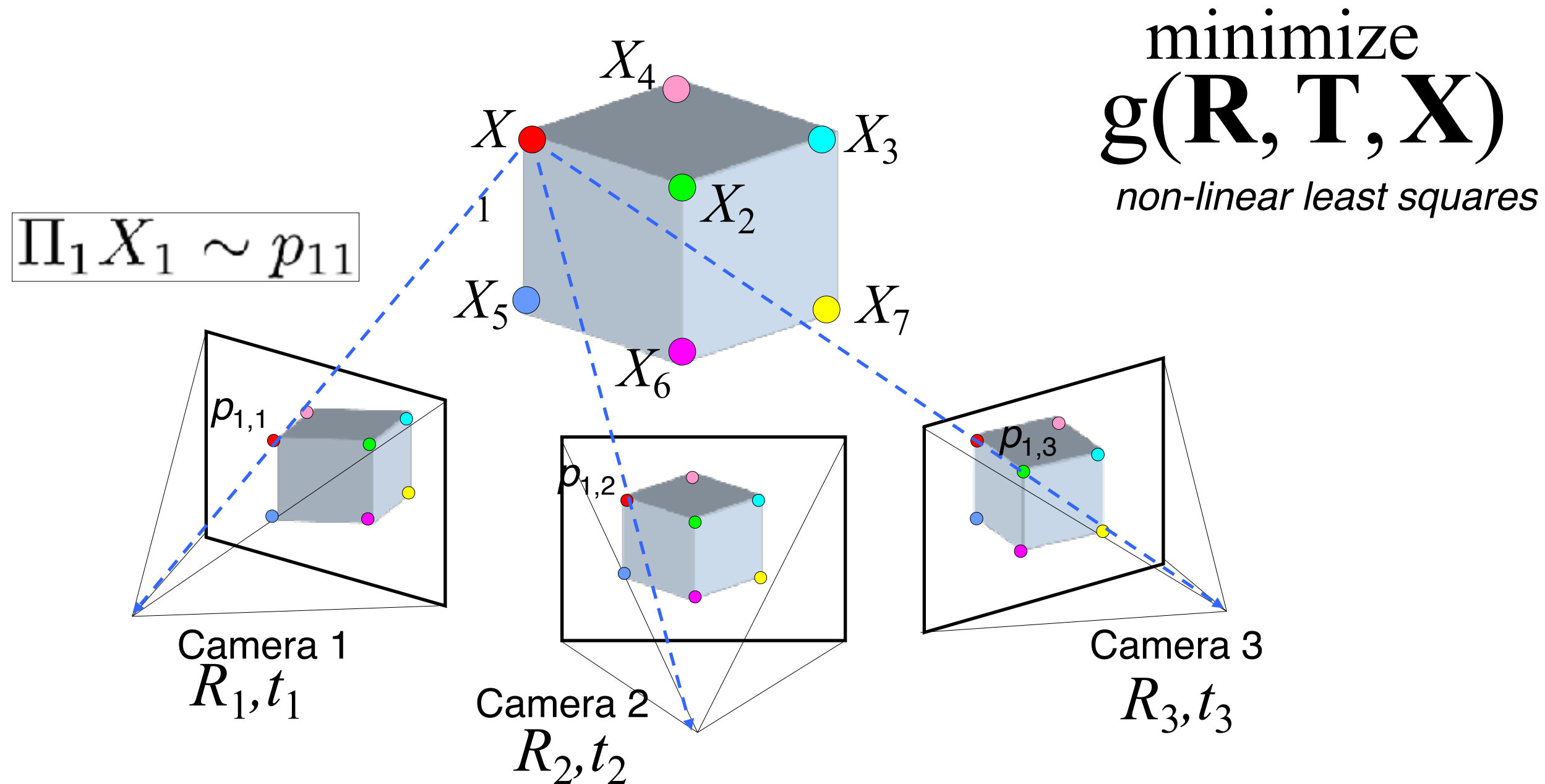
Structure from motion

What if we don't know 3D points or camera parameters?

SfM solves both of these problems *at once*

A kind of chicken-and-egg problem (but solvable)

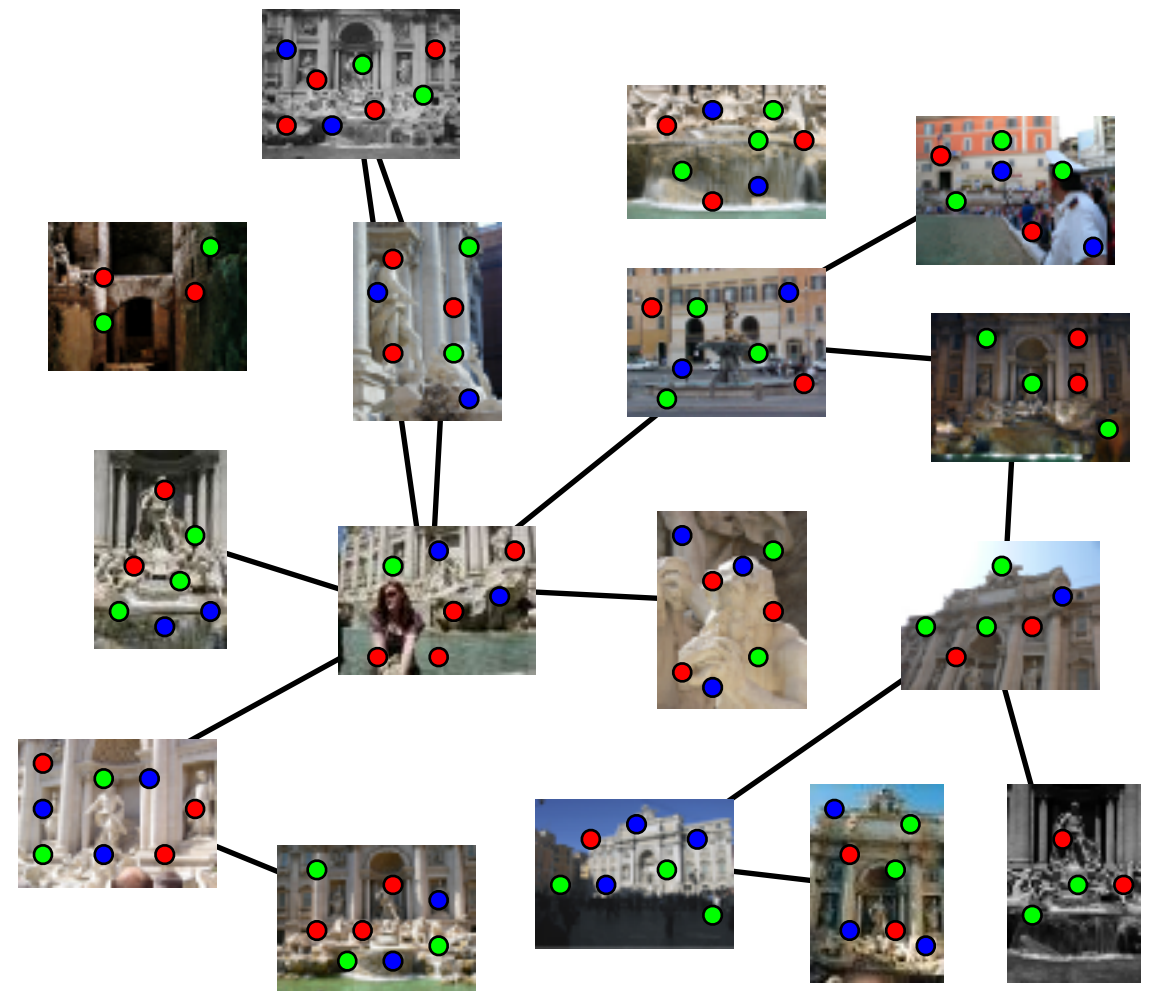
Structure from motion



Feature detection and matching

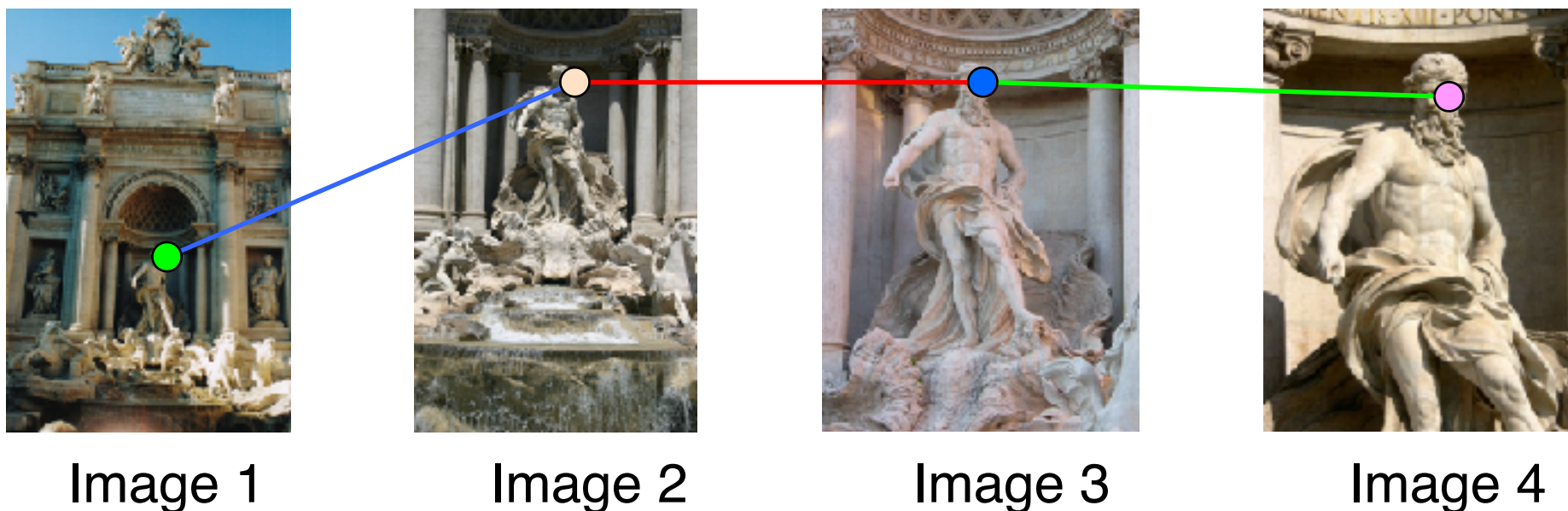
We know how to do that!

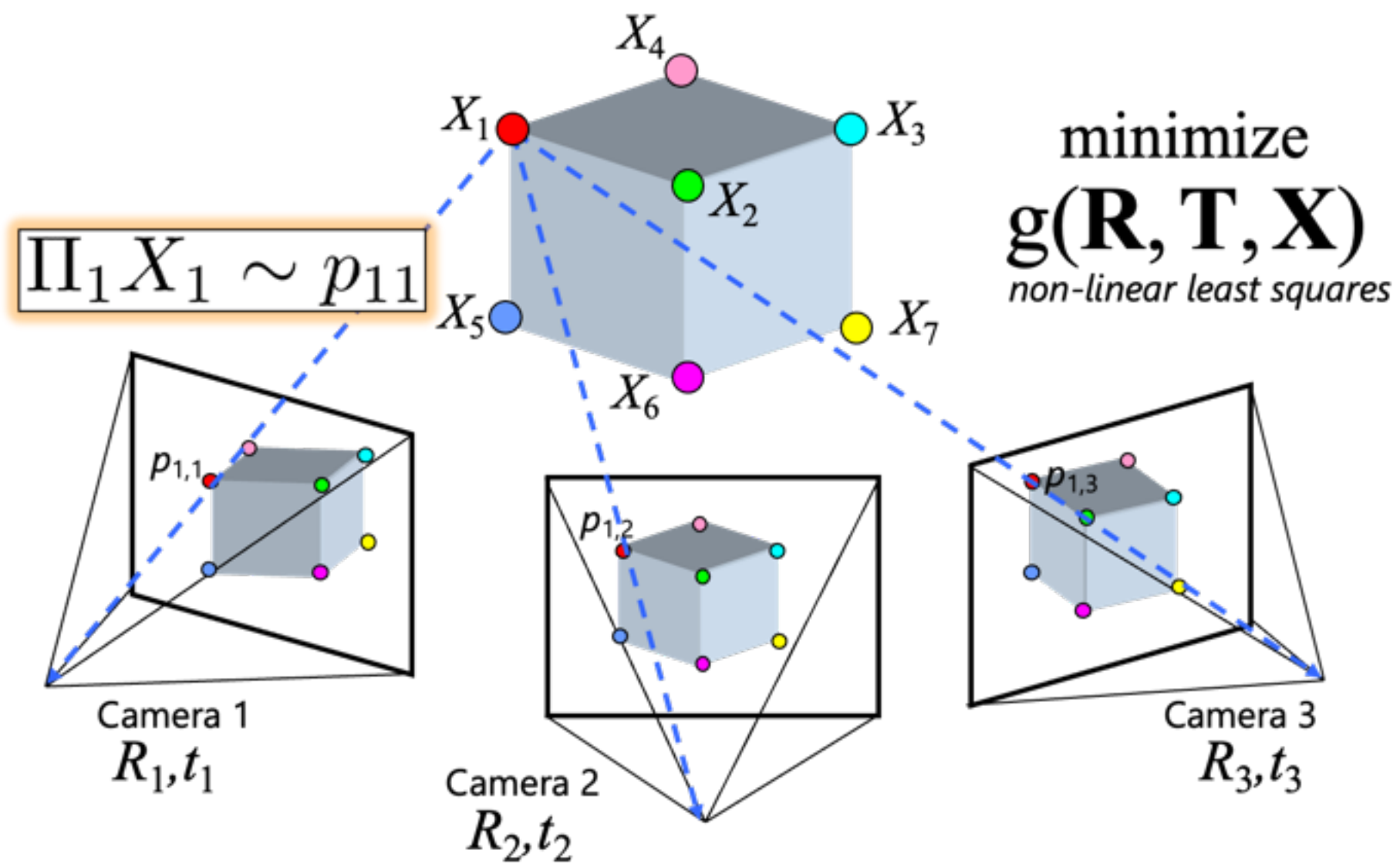
- find and match features between pairs of images
- Remove outliers
- Compute pairwise epipolar geometries



Multiview correspondences

- Link up pairwise matches to form connected components of matches across several images





Problem size

- What are the variables?
 - Cameras and points
- How many variables per camera?
 - 6 (if calibrated), more if uncalibrated
- How many variables per point?
 - 3
- Trevi Fountain collection: very large optimization problem
 - 466 input photos
 - + > 100,000 3D points

Structure from motion

- Minimize sum of squared reprojection errors:

$$g(\mathbf{X}, \mathbf{R}, \mathbf{T}) = \sum_{i=1}^m \sum_{j=1}^n \underbrace{w_{ij}}_{\substack{\text{indicator variable:} \\ \text{is point } i \text{ visible in image } j?}} \cdot \left\| \underbrace{\mathbf{P}(\mathbf{x}_i, \mathbf{R}_j, \mathbf{t}_j)}_{\substack{\text{predicted} \\ \text{image location}}} - \underbrace{\begin{bmatrix} u_{i,j} \\ v_{i,j} \end{bmatrix}}_{\substack{\text{observed} \\ \text{image location}}} \right\|^2$$

- Minimizing this function is called *bundle adjustment*
 - Optimized using non-linear least squares
 - e.g. Levenberg-Marquardt algorithm

Vision Algorithms: Theory and Practice

International Workshop on Vision Algorithms
Corfu, Greece, September 1999
Proceedings

Bundle Adjustment — A Modern Synthesis

Bill Triggs¹, Philip F. McLauchlan², Richard I. Hartley³, and Andrew W. Fitzgibbon⁴

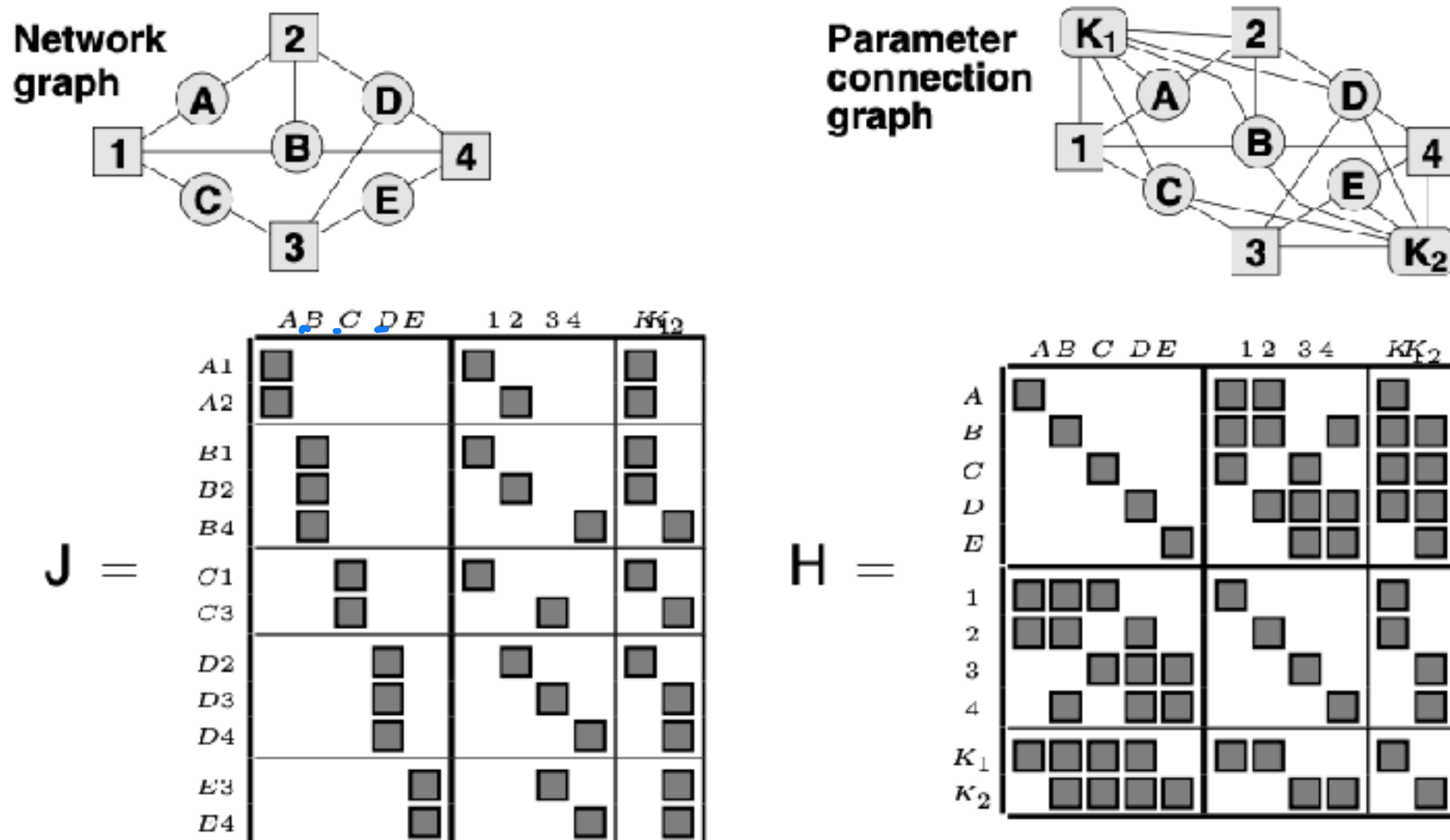


Fig. 3. The network graph, parameter connection graph, Jacobian structure and Hessian structure for a toy bundle problem with five 3D features A–E, four images 1–4 and two camera calibrations K_1 (shared by images 1,2) and K_2 (shared by images 3,4). Feature A is seen in images 1,2; B in 1,2,4; C in 1,3; D in 2–4; and E in 3,4.

Building Rome on a cloudless day (Frahm, Pollefeys et al., ECCV 2010)



PIETA